

FISH DETECTION IN MARINE ENVIRONMENTS USING YOLO-BASED OBJECT DETECTION

A REPORT

*Submitted for Partial fulfillment of the Requirement
For the award of the degree of*

BACHELOR OF TECHNOLOGY

**In
Computer Engineering**

Submitted by

Preet Karani (19)

Sumeet Kedare (20)

Vatsal Lodaya (21)

Shiven Maurya (23)

Under the guidance of

Prof. Krupa Chotai

Assistant/Associate Professor/Professor



**DEPARTMENT OF COMPUTER ENGINEERING
SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE**
(An Autonomous Institute Affiliated to University of Mumbai)
MUMBAI - 400 088, MAHARASHTRA (INDIA)

2025–2026



Shah & Anchor Kutchhi Engineering College
(An Autonomous Institute Affiliated to University of Mumbai)
Mumbai-400 088, MAHARASHTRA (India)

CERTIFICATE

It is my pleasure to certify that **Preet Karani, Sumeet Kedare, Vatsal Lodaya** and **Shiven Maurya** worked under my supervision for the B. Tech. Project entitled **FISH DETECTION IN MARINE ENVIRONMENTS USING YOLO-BASED OBJECT DETECTION** and their work is of the level of requirement set up for the Project in **Computer Engineering** by Shah & Anchor Kutchhi Engineering College, Mumbai.

Prof. Krupa Chotai

Assistant/Associate Professor

Department of Computer Engineering

Shah & Anchor Kutchhi Engineering College

Mumbai – 400 088, India

The Oral and Practical examination of **Preet Karani, Sumeet Kedare, Vatsal Lodaya** and **Shiven Maurya**, B. Tech. in Computer Engineering, has been held on
.....

External Examiner

Internal Examiner

Guide Name

Principal

Head of Department

College Seal

CANDIDATE'S DECLARATION

I hereby declare that the work presented in the Project Report entitled “**FISH DETECTION IN MARINE ENVIRONMENTS USING YOLO-BASED OBJECT DETECTION**” for partial fulfilment of the requirement for the degree of B. Tech. in Computer Engineering, submitted to the Department of Computer Engineering at Shah & Anchor Kutchhi Engineering College, Mumbai, is an authentic record of my own work/cited work carried out during the period from to under the supervision of **Prof. Krupa Chotai**.

The matter presented in this Project Report has not been submitted elsewhere in part or fully to any other University or Institute for the award of any other degree.

Name of the Student	Roll No.	Signature
Preet Karani	19	
Sumeet Kedare	20	
Vatsal Lodaya	21	
Shiven Maurya	23	

Date:

Place:

CERTIFICATE OF PLAGIARISM CHECK

Name of the Student	Preet Karani Sumeet Kedare Vatsal Lodaya Shiven Maurya
Title of the Dissertation/Report	FISH DETECTION IN MARINE ENVIRONMENTS USING YOLO-BASED OBJECT DETECTION
Name of the Guide	Prof. Krupa Chotai
Name of the Department	Department of Computer Engineering
Similar content (%) identified	%
Acceptable Maximum Limit	10%

Name & Sign of the Department Academic	Name & Sign of the Students
---	--

Integrity Panel (DAIP) Member	Preet Karani Sumeet Kedare Vatsal Lodaya Shiven Maurya
--------------------------------------	---

Name & Sign of the Guide: Prof. Krupa Chotai

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to **Prof. Krupa Chotai** for her invaluable guidance, continuous encouragement, and unwavering support throughout the duration of this project. Her expertise in the domain and thoughtful feedback at every stage were instrumental in shaping both the technical depth and the presentation quality of this work.

We extend our heartfelt thanks to the **Department of Computer Engineering** and the management of **Shah & Anchor Kutchhi Engineering College** for providing the computational resources, laboratory facilities, and the conducive academic environment necessary for carrying out this research.

We are also grateful to **Dr. Vidyullata Devmane**, Head of the Department of Computer Engineering, for her administrative support and motivating words throughout the academic year.

Special thanks are due to our fellow students and colleagues whose constructive discussions and suggestions greatly enriched our understanding of deep learning and computer vision techniques applied to marine ecosystems.

Finally, we acknowledge with deep appreciation the constant love, patience, and moral support of our families, without whom this journey would not have been possible.

Preet Karani
Sumeet Kedare
Vatsal Lodaya
Shiven Maurya

Shah & Anchor Kutchhi Engineering College

Date:

ABSTRACT

The monitoring of fish populations across diverse marine environments presents a formidable challenge due to the inherent complexities of the underwater visual domain, including wavelength-dependent colour distortion, backscatter from suspended particles, dynamic lighting gradients, and inter-species occlusion. Traditional monitoring approaches rely predominantly on labour-intensive manual review of underwater footage, which introduces significant throughput limitations and inter-observer inconsistencies.

This project presents an automated, end-to-end fish detection and species classification system built upon two advanced variants of the You Only Look Once (YOLO) architecture — YOLOv9 and YOLOv11. Both models are trained from scratch on a curated multi-species dataset spanning 24 fish species commonly encountered in marine and aquaculture environments, using data augmentation strategies that simulate realistic underwater capture conditions. The system supports both static image inputs and real-time live video stream processing at 30 frames per second.

A user-accessible web application developed with the Python Flask framework provides modules for user authentication, image upload, live detection, and result visualisation. Quantitative evaluation on a held-out test split demonstrates that YOLOv11 achieves a mean Average Precision (mAP50) of 0.70 and YOLOv9 achieves 0.67, with inference speeds of 22 ms and 18 ms per frame respectively. Qualitative inspection confirms accurate bounding-box localisation for both isolated and co-occurring species under varied underwater conditions. All test cases in the functional validation suite passed successfully.

The proposed system addresses the critical shortcomings of existing manual and rudimentary automated monitoring pipelines by delivering real-time throughput, scalable architecture, and an accessible interface designed for non-specialist marine researchers. The findings reinforce the utility of modern YOLO-based detectors for automated aquatic biodiversity monitoring and lay the groundwork for future extensions including edge deployment, domain adaptation, and explainability integration.

Keywords: Fish Detection, YOLOv9, YOLOv11, Object Detection, Marine Ecosystems, Deep Learning, Python Flask, Underwater Image Analysis, Real-Time Detection, Aquatic Monitoring.

Contents

Certificate	ii
Candidate's Declaration	iii
Certificate of Plagiarism Check	iv
Acknowledgement	v
Abstract	vi
1 Introduction	1
1.1 Background	1
1.2 Motivation	2
1.3 Organization of the Report	3
2 Literature Review	4
2.1 Survey of Existing Systems	4
2.2 Limitation of Existing Systems and Research Gap	8
2.3 Problem Statement and Objectives	9
2.4 Scope	10
3 Software Requirement Specification	11
3.1 Introduction	11
3.2 Overall Description	12
3.3 External Interface Requirements	14
3.4 System Features	16
3.4.1 User Authentication Module	16
3.4.2 Image Upload and Prediction Module	16
3.4.3 Live Detection Module	17
3.4.4 Database and Session Module	18
3.5 Other Non-Functional Requirements	18
3.6 Other Requirements	19
4 Project Scheduling and Planning	20
4.1 Introduction	20
4.2 Project Phases	20
4.3 Project Timeline	21
4.4 Resource Allocation	21
4.5 Summary	22

5	Proposed System	23
5.1	Feasibility Study	23
5.2	Analysis / Framework / Algorithm	24
5.3	Details of Hardware and Software	25
5.4	Design Details	25
5.4.1	System Architecture	25
5.4.2	Project Workflow	26
5.4.3	Use Case Diagram	27
5.4.4	Class Diagram	28
5.4.5	Sequence Diagram	29
5.4.6	Collaboration Diagram	30
5.4.7	Deployment Diagram	31
5.4.8	Component Diagram	32
5.4.9	Activity Diagram	33
5.4.10	Entity Relationship Diagram	33
5.4.11	Data Flow Diagram	35
5.5	Methodology	36
6	Implementation Details	38
6.1	Dataset Preparation	38
6.2	Model Training Configuration	39
6.3	Web Application Modules	41
6.3.1	Authentication Module	41
6.3.2	Image Upload Module	41
6.3.3	Static Prediction Module	42
6.3.4	Live Detection Module	42
6.4	Frontend Design	43
6.5	Database Schema	44
7	System Testing	45
7.1	Feasibility and Testing Overview	45
7.2	Types of Testing Conducted	45
7.3	Test Cases and Results	46
8	Results and Discussion	50
8.1	Quantitative Evaluation	50
8.2	Training Convergence Analysis	51
8.3	Qualitative Observations	52
8.4	Web Application Output Screens	53
8.5	Discussion	57

9 Conclusion and Future Scope	58
9.1 Conclusion	58
9.2 Future Enhancement	59
Bibliography	61
Appendix	62

List of Figures

5.1	System Architecture Diagram	26
5.2	Project Workflow Diagram	27
5.3	Use Case Diagram – System Actor	28
5.4	Use Case Diagram – User Actor	28
5.5	Class Diagram	29
5.6	Sequence Diagram	30
5.7	Collaboration Diagram	31
5.8	Deployment Diagram	32
5.9	Component Diagram	32
5.10	Activity Diagram	33
5.11	Entity Relationship (ER) Diagram	34
5.12	Data Flow Diagram – Level 1	35
5.13	Data Flow Diagram – Level 2	36
8.1	YOLOv11 Training and Validation Metrics (150 Epochs)	51
8.2	YOLOv9 Training and Validation Metrics (150 Epochs)	52
8.3	Index Page – Fish Detection System Landing Page	53
8.4	Registration Page – Create Account Form	54
8.5	Login Page – User Authentication	54
8.6	Home Dashboard – Post-Login Navigation Page	55
8.7	About Page – System Description	55
8.8	Prediction Page – Image Upload and Fish Species Detection	55
8.9	Live Detection Page – Real-Time Webcam Fish Detection	56
8.10	Result Page – Detection Results with Bounding Box and Confidence Score	56
8.11	Live Detection Window – Common Silver Carp Detected at 30 FPS with Confidence 0.97	57

List of Tables

2.1	Summary of Research Papers on AI-Based Fish Detection (2019–2025)	7
4.1	Project Phase Timeline	21
4.2	Team Member Responsibilities	22
5.1	Hardware and Software Specifications	25
6.1	Model Training Hyperparameters	39
6.2	MySQL Database Tables	44
7.1	Functional Test Cases and Results	47
8.1	Comparative Performance of YOLOv9 and YOLOv11 on the Test Set .	50

Chapter 1

Introduction

1.1 Background

The oceans and freshwater bodies of our planet are home to an estimated 30,000 documented fish species, collectively underpinning global food security, ecological stability, and the livelihoods of hundreds of millions of people engaged in fishing and aquaculture. Yet these ecosystems face an unprecedented combination of anthropogenic stressors — overfishing, water pollution, habitat degradation, and accelerating climate change — that have driven numerous species towards population collapse or extinction.

A particularly insidious problem within commercial and artisanal fisheries is the unintentional capture of juvenile or premature fish before they reach reproductive maturity. These early-stage individuals are morphologically similar across species, making accurate identification with the naked eye extremely difficult, especially under time pressure in the field. When young fish are harvested en masse, natural population recovery is disrupted, pushing vulnerable species further toward decline.

Traditional fish monitoring in both wild and aquaculture environments has relied on diver-conducted transect surveys, baited remote underwater video (BRUV) stations, and manual annotation of camera-trap footage. Each of these approaches is labour-intensive, geographically constrained, prone to inter-observer subjectivity, and fundamentally incompatible with the continuous, large-scale monitoring that modern conservation demands.

The emergence of deep learning-powered computer vision — and in

particular the family of YOLO (You Only Look Once) single-pass object detectors — offers a compelling opportunity to automate fish species recognition with a throughput and consistency that far exceeds human capacity. By training detection models on sufficiently large and diverse annotated datasets, it is possible to build systems that identify and localise fish within an image in milliseconds, making real-time monitoring a practical reality.

1.2 Motivation

Our motivation for this project arises from two converging imperatives: the ecological urgency of protecting marine biodiversity, and the engineering opportunity presented by modern deep learning to address this challenge at scale.

Fishermen and marine researchers frequently encounter difficulty distinguishing between species — especially juvenile fish — in their early developmental stages. An automated identification system could assist them in making better decisions during fishing operations, thereby reducing the capture of endangered or legally protected species. Such a system could equally support government fisheries agencies and conservation NGOs in monitoring aquatic ecosystems far more efficiently than current methods permit.

From a technical standpoint, the YOLO architecture family has matured considerably through successive versions. YOLOv9 and YOLOv11, the two models investigated in this work, incorporate architectural refinements — including attention mechanisms, improved feature pyramid networks, and anchor-free regression heads — that make them well-suited to the challenges of underwater imagery: variable lighting, turbidity, and the fine-grained inter-species visual similarities that characterise fish morphology.

The project therefore aims to bridge the gap between ecological need and technological capability by delivering a trained, deployed, and vali-

dated fish detection system accessible to non-specialist users through a web-based interface.

1.3 Organization of the Report

This report is organised as follows:

- **Chapter 2** presents a survey of existing research in AI-based fish detection, identifies key limitations in the current literature, and formally states the problem objectives and project scope.
- **Chapter 3** defines the Software Requirements Specification (SRS) for the proposed system, covering functional and non-functional requirements, system features, and interface specifications.
- **Chapter 4** describes the project scheduling, phase-wise planning, resource allocation, and timeline.
- **Chapter 5** presents the proposed system design, including feasibility analysis, system architecture, data flow, UML diagrams, and methodology.
- **Chapter 6** details the full implementation of the system, covering dataset preparation, model training configuration, web application modules, and code excerpts.
- **Chapter 7** documents the system testing strategy and presents the functional test case results.
- **Chapter 8** presents quantitative and qualitative evaluation results for both YOLOv9 and YOLOv11.
- **Chapter 9** concludes the report with a summary of findings and directions for future work.

Chapter 2

Literature Review

2.1 Survey of Existing Systems

Over the last decade, the application of deep learning to underwater fish detection and species classification has attracted growing scholarly attention. The YOLO family of single-pass object detectors has emerged as the dominant paradigm owing to their single-pass inference speed and competitive accuracy on standard benchmarks.

CNN-Based Classification (2019)

Li and Wang [1] proposed a Convolutional Neural Network (CNN) model to classify fish species from underwater images. The model achieved 92% classification accuracy on their benchmark dataset, demonstrating that deep learning could automate fish identification at a level suitable for conservation applications. However, the system was trained exclusively on adult fish images and did not address juvenile or premature stages.

YOLO-Based Real-Time Detection (2020)

Ahmed and Yousaf [2] employed YOLOv3 for real-time fish detection in low-light underwater environments. Their system demonstrated fast inference speeds and highlighted the ability of YOLO architectures to monitor endangered species in natural habitats. The study confirmed that single-stage detectors were practical for field deployment, though accuracy degraded markedly under extreme turbidity.

Juvenile Fish Identification (2021)

Sharma and Kumar [3] built a CNN model specifically targeting premature and juvenile fish stages, with the explicit goal of monitoring survival rates in aquaculture settings. Their work was among the first to focus on stage-specific classification rather than simple species identification, directly addressing the conservation problem of premature fish capture. Accuracy on juvenile stages, however, remained lower than on adult specimens owing to limited annotated data.

AI-Driven Biodiversity Monitoring (2022)

Tanaka and Mori [4] applied machine learning methods to analyse biodiversity patterns and track endangered fish species across large geographic areas. Their scalable system demonstrated meaningful capacity to assess extinction risk and population decline from aggregated camera data, establishing a framework for longitudinal ecological monitoring.

Lightweight Underwater Detection — LFN-YOLO (2025)

Liu et al. [5] proposed LFN-YOLO, a detector tailored for underwater small-object scenarios. The architecture employs a re-parameterised backbone based on RepGhost modules to reduce computational overhead, integrates Spatial-to-Depth convolutions (SPD-Conv) for fine-grained feature preservation, and adopts a Generalised Feature Pyramid Network (GFPN) for multi-scale representation. Evaluated on the URPC, Brackish, and TrashCan benchmarks, LFN-YOLO attains competitive mAP@0.5 with approximately 2.7 million parameters and 7.2 GFLOPs, demonstrating near-edge deployability.

AquaYOLO for Aquaculture (2025)

Vijayalakshmi et al. [6] introduced AquaYOLO, an adaptation of the YOLO paradigm targeted at aquaculture pond environments. Their architecture incorporates CSP-based convolutional blocks tuned for hierarchical feature extraction under variable water clarity and lighting. The detector was validated on the DePondFi dataset comprising over 50,000 bounding-box annotations from 8,000 images collected across South Indian aquaculture sites, showing reliable species localisation under realistic pond conditions.

Detection Under Degraded Conditions (2024)

Chen et al. [7] tackled the problem of training object detectors under degraded or incomplete prior knowledge — a common scenario when underwater imagery suffers from severe noise or distortion. Their adaptive training strategy improves model robustness when ground-truth annotation quality is limited, yielding meaningful detection accuracy even in highly degraded visual environments.

Real-Time Classification with YOLOv5 (2024)

Sharma and Gupta [8] demonstrated real-time, multi-species fish classification using YOLOv5 with an ONNX inference backend, achieving an F1-score of 0.87 across six species at approximately 30 frames per second, confirming that YOLO variants can support continuous live-stream monitoring under controlled conditions.

Summary of Reviewed Literature

Table 2.1: Summary of Research Papers on AI-Based Fish Detection (2019–2025)

Sr.	Paper & Year	Authors	Summary	Outcome
1	Deep Learning-based Fish Species Classification Using CNN (2019)	Li & Wang	CNN to classify fish species from underwater images	92% accuracy
2	YOLO-based Real-Time Fish Detection (2020)	Ahmed & Yousaf	YOLOv3 for real-time fish detection in low-light environments	Fast detection, endangered species monitoring
3	Automatic Fry and Juvenile Fish Identification (2021)	Sharma & Kumar	CNN for premature/juvenile fish identification	Helps prevent early extinction
4	AI-Driven Biodiversity Monitoring (2022)	Tanaka & Mori	ML to track endangered fish and biodiversity	Scalable extinction risk analysis
5	LFN-YOLO (2025)	Liu et al.	Lightweight re-parameterised detector for small underwater objects	Competitive mAP, edge-deployable
6	AquaYOLO (2025)	Vijayalakshmi et al.	YOLO adapted for aquaculture pond environments	Reliable detection in pond conditions
7	Degraded Prior Knowledge Detection (2024)	Chen et al.	Adaptive training for noisy underwater imagery	Robust detection under low quality inputs

Table 2.1 – continued

Sr.	Paper & Year	Authors	Summary	Outcome
8	YOLOv5 Real-Time Classification (2024)	Sharma & Gupta	Real-time multi-species classification at 30 FPS	F1-score 0.87

2.2 Limitation of Existing Systems and Research Gap

Despite significant advances, several critical limitations persist across the reviewed literature:

- Many publicly available fish datasets contain images predominantly of adult specimens with well-defined morphological features. Models trained on such data generalise poorly to juvenile or premature stages, which are visually ambiguous.
- Most existing automated systems either operate on static images or handle a narrow set of species. Very few systems offer an integrated pipeline that supports both still-image and live-video detection for a broad species spectrum.
- There is a notable scarcity of comparative deployments of multiple advanced YOLO variants — particularly YOLOv9 and YOLOv11 — within an integrated, user-accessible web platform designed for generalist marine monitoring.
- Few systems link detection results to actionable conservation meta-data such as edibility, conservation status, or population risk, limiting their practical utility for fishermen and researchers.

2.3 Problem Statement and Objectives

Problem Statement

Monitoring fish populations in marine environments is a complex task due to the low visibility of underwater footage, diverse species morphology, and rapidly changing environmental conditions. Traditional fish detection methods require significant manual effort, leading to inefficiencies and inaccuracies. Existing automated detection systems struggle to accurately identify fish in real-world underwater environments due to lighting variations, turbid water, motion blur, and overlapping species. There is therefore a need for a more efficient and robust system capable of accurately detecting fish species from underwater images and videos at real-time speeds, with minimal manual intervention.

Objectives

1. Train YOLOv9 and YOLOv11 models on a diverse annotated dataset spanning 24 fish species to ensure accurate detection and species classification.
2. Develop a fully operational Flask-based web application incorporating modules for user authentication, image submission, and live video stream processing.
3. Evaluate both models quantitatively using standard detection metrics including precision, recall, mAP50, and mAP50-95.
4. Optimise inference pipelines for real-time responsiveness suitable for continuous ecological monitoring.
5. Deliver a transparent and interpretable prediction interface that equips marine researchers with actionable species information including conservation alerts.

6. Reduce human dependency in fieldwork and improve decision-making for sustainable and responsible fishing practices.

2.4 Scope

The scope of this project covers the end-to-end development of a fish detection system from dataset curation and model training through to web-based deployment. Specifically:

- Early-stage and adult fish identification using deep learning (YOLOv9 and YOLOv11).
- Integration with a structured dataset containing species name, edibility status, and conservation category.
- A front-end web interface for uploading images and streaming live camera feeds, with annotated detection results.
- Both static image prediction and real-time live detection at 30 FPS.
- The system is intended for research, conservation awareness, fisheries management, and educational purposes.

Chapter 3

Software Requirement Specification

3.1 Introduction

This Software Requirements Specification (SRS) is prepared in accordance with the IEEE 830-1984 standard [9]. It defines the functional and non-functional requirements for the AI-powered Fish Detection System, which leverages YOLOv9 and YOLOv11 deep learning models to identify fish species in underwater images and live video streams.

Purpose

The purpose of this document is to provide a comprehensive and unambiguous specification of the system's behaviour, inputs, outputs, and expected operating environment, serving as a reference for all development, testing, and deployment activities.

Document Conventions

This document uses standard IEEE SRS conventions. Requirements are identified with unique identifiers of the form REQ-N. Priority levels are designated High (H), Medium (M), or Low (L).

Intended Audience and Reading Suggestions

This document is intended for the development team, academic supervisors, internal and external examiners, and future students extending this

work. Readers should begin with the overall description (Section 3.2) before proceeding to detailed feature specifications (Section 3.4).

Product Scope

The Fish Detection System is a standalone web-based application that accepts fish images or live video as input and returns species identification, confidence scores, and conservation status alerts. It targets marine biologists, fisheries researchers, conservation officers, fishermen, and students in marine studies.

3.2 Overall Description

Product Perspective

The proposed software operates as a client-server web application. The frontend is served via Flask and accessed through a standard web browser. The backend orchestrates data ingestion, preprocessing, model inference (YOLOv9 or YOLOv11), and result persistence in a MySQL database. The system is self-contained and does not depend on any external third-party prediction API.

Product Functions

- User registration and authenticated login.
- Image upload and validation (JPEG, PNG, BMP formats).
- Real-time live video detection via webcam at 30 FPS.
- Fish species detection and bounding-box annotation using YOLOv9 and YOLOv11.
- Display of species name, detection confidence score, edibility status, and conservation alert.

- Secure session management and logout.
- Storage of prediction results linked to user accounts in MySQL.

User Classes and Characteristics

- **Fishermen and conservation officers:** Non-technical users who need simple upload-and-identify workflows with clear alerts.
- **Marine biologists and researchers:** Users who require detailed confidence scores and species metadata for ecological studies.
- **Students and educators:** Users engaged in species learning and citizen science programmes.
- **System administrators:** Technical users responsible for model re-training and deployment maintenance.

Operating Environment

The application runs on a server with Python 3.9+, Flask, PyTorch, and Ultralytics YOLO installed. Minimum hardware requirements include an Intel Core i3 processor, 8 GB RAM, and 128 GB storage. GPU acceleration (CUDA-enabled) is recommended for model training; CPU inference is supported for deployment. Clients require only a modern web browser and an internet connection (or local network access).

Design and Implementation Constraints

- All deep learning components must use open-source frameworks (PyTorch, Ultralytics) to avoid licensing costs.
- Image inputs are resized to 640×640 pixels prior to inference.
- The live video module operates over a local network; bandwidth-constrained deployments may require frame-rate reduction.

- User credentials are stored using bcrypt hashing; plaintext password storage is prohibited.

Assumptions and Dependencies

- The annotated training dataset is available from publicly accessible sources (Kaggle, Fish4Knowledge).
- XAMPP provides the MySQL database backend for local deployment.
- The deployment server has network access during initial model weight download (pre-trained ImageNet weights).

3.3 External Interface Requirements

User Interfaces

The web interface provides the following screens:

- **Index / Landing Page:** Public entry point with navigation to Login and Register.
- **Registration Page:** Form collecting full name, email, and password with confirmation.
- **Login Page:** Email and password form with session creation on success.
- **Home Dashboard:** Post-login landing page with links to Prediction and Live Detection modules.
- **Prediction Page:** Drag-and-drop or click-to-upload image zone; “Detect Fish” trigger button; annotated result display with species name, confidence, and conservation status.
- **Live Detection Page:** Camera activation control; real-time MJPEG stream display with overlaid bounding boxes and species labels.

- **About Page:** Project description and technology overview.

Alert colours on prediction output:

- **Green:** Safe to catch (non-endangered species).
- **Orange:** Vulnerable species — handle with caution.
- **Red:** Endangered or legally protected species — do not catch.

Hardware Interfaces

The system interfaces with standard web cameras (USB or integrated) for live detection. No specialised underwater camera hardware is required; images captured by any device may be uploaded for static prediction.

Software Interfaces

- **PyTorch / Ultralytics YOLO:** Model training and inference.
- **OpenCV:** Image preprocessing and live video frame capture.
- **Flask / Flask-Login:** Web server framework and session management.
- **MySQL (via XAMPP):** Persistent storage of user and prediction data.
- **NumPy, Pandas:** Data manipulation and augmentation utilities.

Communications Interfaces

The web application communicates over HTTP/HTTPS. Live video is streamed as a multipart MJPEG response. All form submissions use standard HTTP POST requests.

3.4 System Features

3.4.1 User Authentication Module

Description and Priority

High priority. Manages user registration with hashed credential storage (bcrypt), login session creation, and logout. Without authentication, access to prediction features is blocked.

Functional Requirements

- REQ-1: The system shall accept user registration with name, email, and password.
- REQ-2: Passwords shall be stored using bcrypt hashing; plaintext storage is prohibited.
- REQ-3: The system shall validate email format and reject duplicate registrations.
- REQ-4: The system shall create a persistent session on successful login.
- REQ-5: The system shall redirect unauthenticated requests to the login page.

3.4.2 Image Upload and Prediction Module

Description and Priority

High priority. Core functional feature; allows users to submit fish images for species detection.

Functional Requirements

- REQ-6: The system shall accept image files in JPEG, PNG, and BMP formats.
- REQ-7: Uploaded images shall be resized to 640×640 pixels and normalised prior to inference.
- REQ-8: The system shall display the detected species name, confidence score, and conservation status within 3 seconds.
- REQ-9: The system shall overlay annotated bounding boxes on the result image.
- REQ-10: The system shall display a colour-coded conservation alert (green / orange / red).

3.4.3 Live Detection Module

Description and Priority

High priority. Enables real-time fish monitoring via webcam, directly supporting ecological field use.

Functional Requirements

- REQ-11: The system shall capture webcam frames via OpenCV and process them at up to 30 FPS.
- REQ-12: The system shall stream annotated MJPEG frames to the browser with bounding boxes and species labels.
- REQ-13: The user shall be able to start and stop the live stream from the web interface.

3.4.4 Database and Session Module

Description and Priority

Medium priority. Ensures result persistence and session continuity.

Functional Requirements

- REQ-14: The system shall store prediction results (species, confidence, image path) linked to the authenticated user.
- REQ-15: Sessions shall expire on logout or browser closure.
- REQ-16: The database schema shall include `USERS`, `IMAGES`, and `PREDICTIONS` tables.

3.5 Other Non-Functional Requirements

Performance Requirements

- Static image prediction shall complete within 3 seconds on CPU; under 1 second on GPU.
- Live detection shall sustain a minimum of 15 FPS on CPU hardware and 30 FPS on GPU.
- Web page load time shall not exceed 4 seconds on a standard broadband connection.

Security Requirements

- User passwords shall never be stored in plaintext.
- Uploaded files shall be validated for type and size before processing.
- Session tokens shall be invalidated on logout.

Software Quality Attributes

- **Usability:** The interface shall be navigable by non-technical users with no training.
- **Scalability:** The model can be retrained with additional species without structural changes to the application.
- **Maintainability:** All modules are independently deployable and testable.
- **Portability:** The application runs on Windows 10/11, Ubuntu 20.04+, and macOS.

Business Rules

Only authenticated users may access the prediction and live detection features. Model weights and training data are maintained by the system administrators and are not accessible to end users.

3.6 Other Requirements

Documentation will include an installation guide, dataset structure description, model retraining guidelines, and API endpoint specifications for future developers extending the system.

Chapter 4

Project Scheduling and Planning

4.1 Introduction

The project was divided into logical phases across two semesters to ensure systematic development. Semester 7 covered requirements gathering, literature review, dataset study, model prototyping, and the design of the proposed system. Semester 8 addressed full model training, web application development, integration, testing, and final deployment.

4.2 Project Phases

1. **Requirement Gathering** – Understanding the conservation problem, reviewing existing systems, and compiling dataset requirements.
2. **Dataset Preparation** – Collecting, cleaning, annotating, and augmenting underwater fish images spanning 24 species.
3. **Model Training** – Training YOLOv9 and YOLOv11 on the prepared dataset; hyperparameter tuning.
4. **Web Application Development** – Building the Flask backend and HTML/CSS/JS frontend with all required modules.
5. **Integration** – Connecting the YOLO inference pipeline with the Flask web server and MySQL database.
6. **Testing and Validation** – Conducting unit, integration, functional, and acceptance testing.

7. **Deployment and Documentation** – Packaging for deployment, writing technical documentation, and preparing the final report.

4.3 Project Timeline

Table 4.1: Project Phase Timeline

Phase	Description	Duration
1	Requirement gathering and dataset study	2 weeks
2	Data collection, annotation, and augmentation	3 weeks
3	Model training (YOLOv9 and YOLOv11)	4 weeks
4	Web application development (Flask + frontend)	3 weeks
5	System integration and database setup	2 weeks
6	Testing, validation, and bug fixing	2 weeks
7	Deployment, documentation, and final report	2 weeks

4.4 Resource Allocation

Hardware Resources

- GPU-enabled workstation or cloud compute instance (NVIDIA GPU recommended for training).
- Standard development laptops (Intel Core i5/i7, 16 GB RAM) for web development and testing.
- Webcam for live detection testing.

Software Resources

- Python 3.9+, PyTorch, Ultralytics YOLO (YOLOv9 and YOLOv11).
- Flask, Flask-Login, OpenCV, MySQL (XAMPP), Pandas, NumPy, Seaborn.
- Visual Studio Code (IDE), GitHub (version control).
- Roboflow / LabelImg (image annotation).

Team Role Assignments

Table 4.2: Team Member Responsibilities

Team Member	Primary Responsibility
Preet Karani (65)	Frontend development (HTML/CSS/JS) and UI design
Sumeet Kedare (66)	Dataset preparation, annotation, and augmentation
Vatsal Lodaya (67)	Model training, evaluation, and inference pipeline
Shiven Maurya (69)	Flask backend, database integration, documentation

4.5 Summary

The structured two-semester plan ensured balanced progress across research, implementation, and testing phases. Early separation of dataset preparation from model training allowed parallel workstreams, and the modular Flask architecture facilitated independent frontend and backend development, reducing inter-team dependencies and enabling early detection of integration issues.

Chapter 5

Proposed System

5.1 Feasibility Study

Economic Feasibility

The proposed system relies entirely on open-source frameworks (PyTorch, Ultralytics YOLO, Flask, MySQL), eliminating all licensing expenditure. Publicly available datasets from Kaggle and Fish4Knowledge remove the cost of proprietary data acquisition. GPU compute required for model training can be supplied through cloud credits (Google Colab, Kaggle Notebooks) at minimal or zero cost for academic use. Operational inference costs on CPU hardware are negligible, making the system affordable for research institutions in developing regions.

Technical Feasibility

Both YOLOv9 and YOLOv11 are mature, well-documented architectures with active community support through the Ultralytics library. The Flask framework provides robust tooling for rapid web service development. The integration of these components into a coherent detection pipeline is technically well within the capabilities of a team with standard machine learning proficiency. The modular design allows incremental development and straightforward debugging.

Social Feasibility

The web interface is intentionally designed for accessibility by domain scientists without deep computer vision expertise. The registration-based access model ensures user accountability. The interpretable prediction display — featuring annotated images, confidence percentages, and colour-coded conservation alerts — promotes trust among end users, particularly fishermen and conservation officers who are the primary beneficiaries of the system.

5.2 Analysis / Framework / Algorithm

The proposed system implements an AI-based framework combining object detection and species classification to identify fish in both static images and live video streams. The core pipeline consists of four stages:

1. **Image Preprocessing:** Input images are resized to 640×640 pixels, normalised to the $[0, 1]$ range, and optionally enhanced for contrast in turbid conditions.
2. **Object Detection:** The YOLO model (YOLOv9 or YOLOv11) performs single-pass object detection, producing bounding-box coordinates, class probabilities, and objectness scores for all detected fish in the frame.
3. **Species Classification:** The YOLO classification head assigns each detected bounding box a species label from the 24-class taxonomy used during training.
4. **Metadata Lookup and Alert Generation:** The predicted species label is cross-referenced with a structured CSV database containing edibility status and IUCN conservation category. If the species is classified as vulnerable or endangered, a colour-coded alert is generated.

5.3 Details of Hardware and Software

Table 5.1: Hardware and Software Specifications

Component	Specification
Programming Language	Python 3.9+
Deep Learning Framework	PyTorch; Ultralytics YOLO
Web Framework	Flask, Flask-Login
Computer Vision Library	OpenCV 4.x
Data Processing	NumPy, Pandas, Scikit-learn, Seaborn
Frontend	HTML5, CSS3, Bootstrap 5, JavaScript
Database	MySQL (via XAMPP)
IDE	Visual Studio Code
Dataset	Kaggle Fish Species Image Dataset (Mahmood Yousaf)
Minimum Hardware	Intel Core i3, 8 GB RAM, 128 GB Storage
Recommended Hardware	NVIDIA GPU (CUDA), 16 GB RAM, SSD Storage

5.4 Design Details

5.4.1 System Architecture

The system follows a four-layer client-server architecture consisting of (1) an Input Layer accepting fish images from the user via upload or webcam, (2) a Processing Layer performing AI-based analysis using the trained YOLO model, (3) a Database Layer storing credentials and predictions, and (4) an Output Layer rendering annotated images with bounding boxes and conservation status alerts.

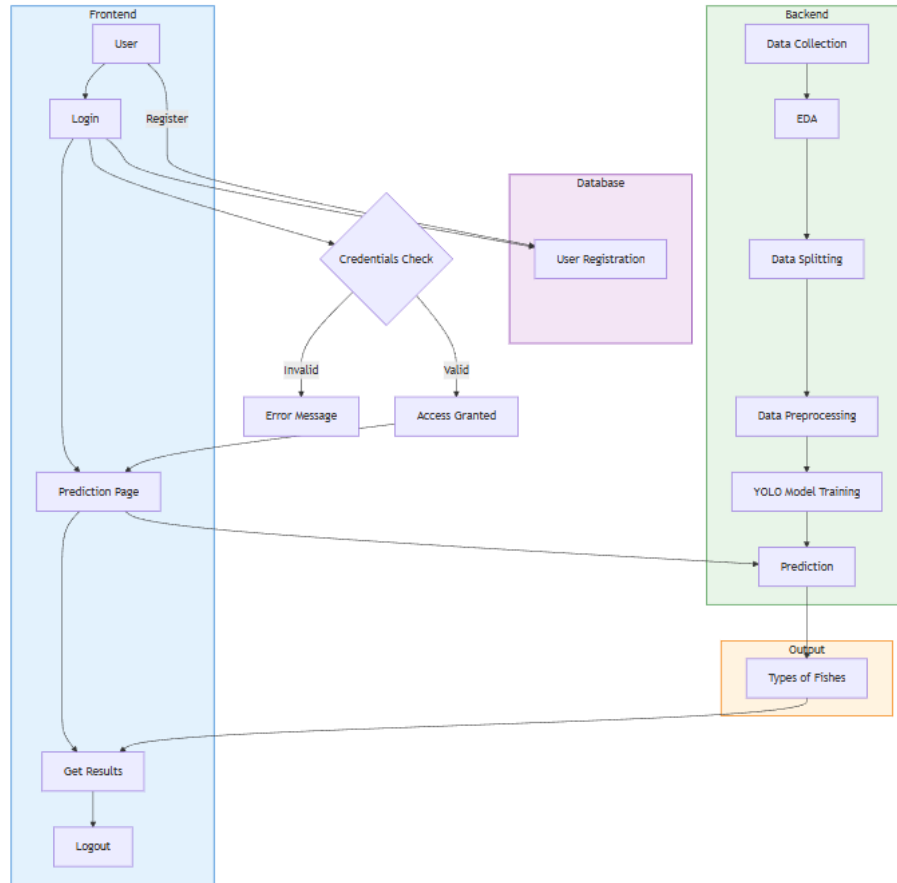


Figure 5.1: System Architecture Diagram

5.4.2 Project Workflow

The overall project workflow illustrates the parallel execution of the User flow (Registration, Login, Input, Results, Logout) and the System back-end flow (Data Collection, Cleaning, Splitting, Preprocessing, Training, Prediction).

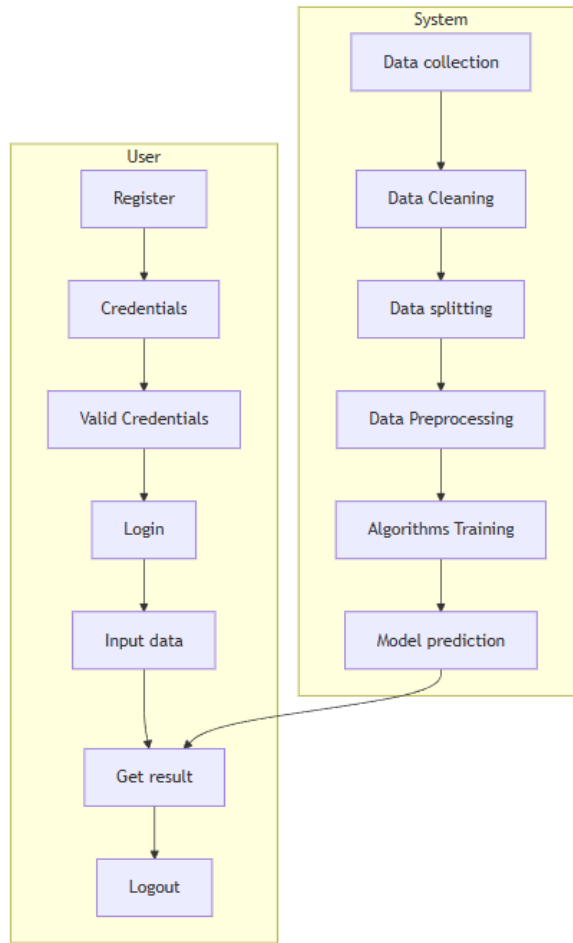


Figure 5.2: Project Workflow Diagram

5.4.3 Use Case Diagram

A use case diagram presents a graphical overview of the functionality provided by a system in terms of actors, their goals, and dependencies between use cases. The system recognises two principal actors: **User** and **System**. The User interacts with modules for Registration, Login, Image Upload, Live Detection, Result Viewing, and Logout. The System pipeline covers Data Collection, Preprocessing, Model Training, Model Evaluation, and Prediction.

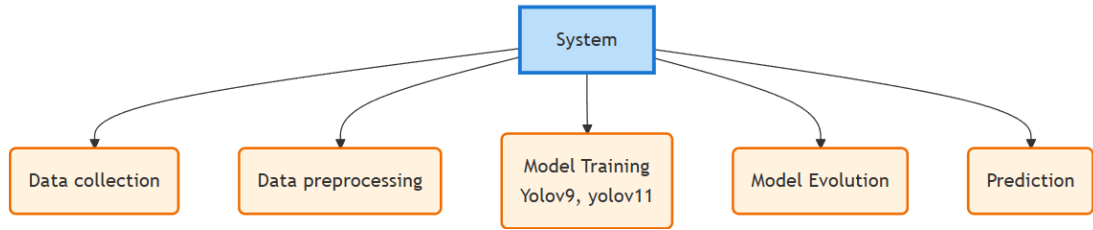


Figure 5.3: Use Case Diagram – System Actor

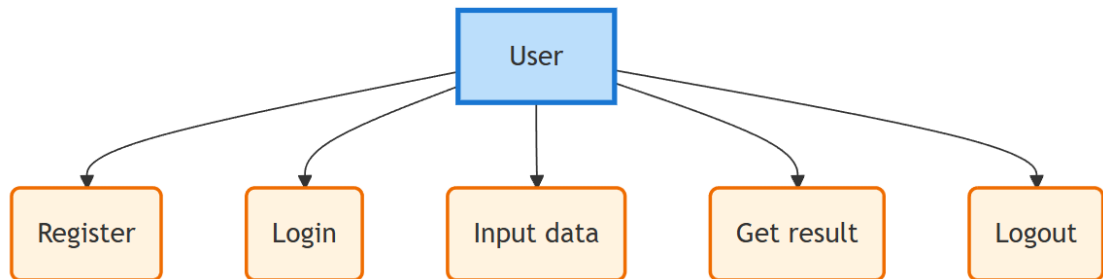


Figure 5.4: Use Case Diagram – User Actor

5.4.4 Class Diagram

A class diagram in UML is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations, and the relationships among the classes. The primary classes are:

- **User** – attributes: `username`, `password`, `email`; methods: `register()`, `login()`, `logout()`.
- **ImageUpload** – attributes: `imagePath`; methods: `uploadImage()`, `getImageDetails()`.
- **Prediction** – attributes: `model` (YOLOv9/YOLOv11); methods: `processImage()`, `getPredictionResults()`.
- **Database** – methods: `storeUserData()`, `storePredictionResults()`.

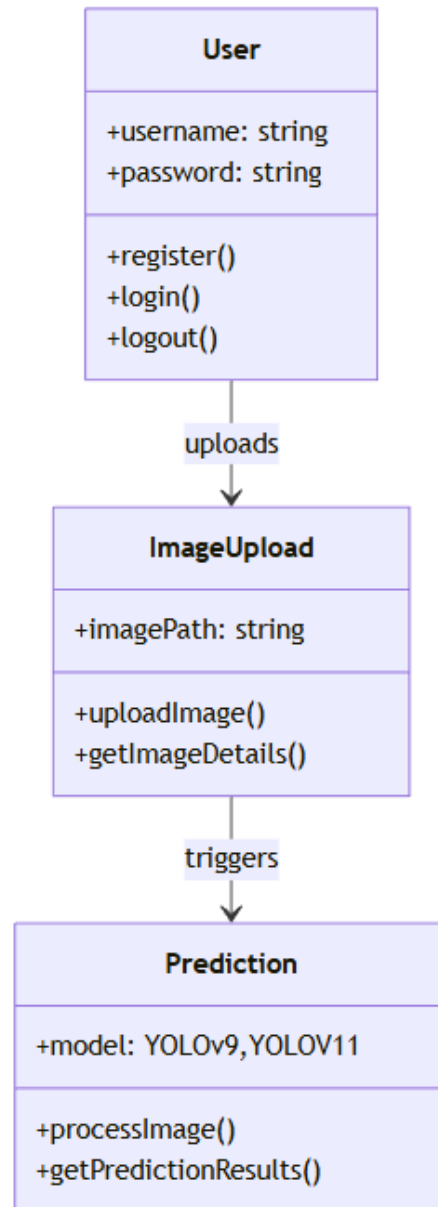


Figure 5.5: Class Diagram

5.4.5 Sequence Diagram

A sequence diagram shows how processes operate with one another and in what order. A typical image-based detection session proceeds as follows: User uploads an image → Flask server receives and validates the file → Preprocessing pipeline resizes and normalises the image → YOLO model performs inference and returns bounding boxes and class scores → Results are stored in the database and returned to the Flask server → Annotated

result is rendered and displayed to the User.

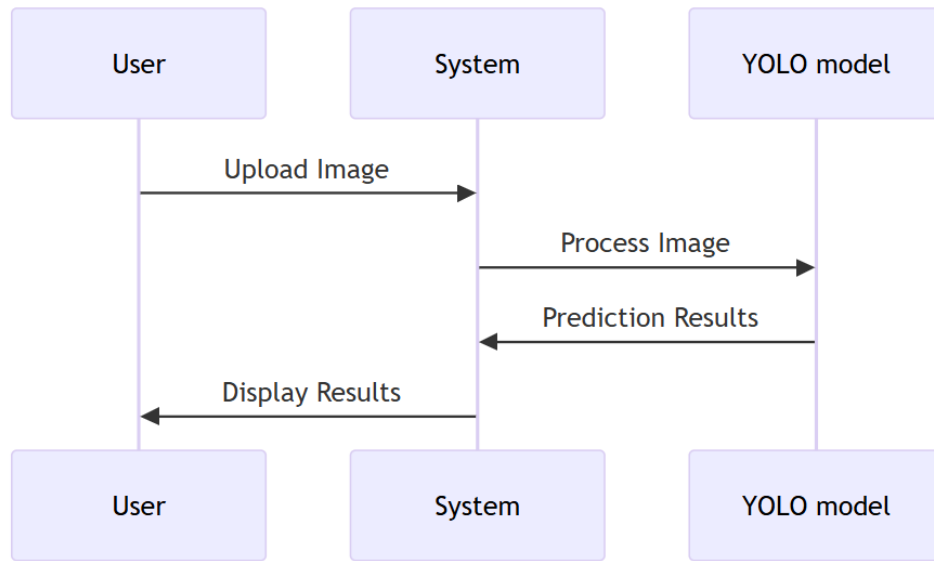


Figure 5.6: Sequence Diagram

5.4.6 Collaboration Diagram

The collaboration diagram illustrates how objects in the system interact and collaborate to produce the desired functionality. Method call sequences are indicated by numbering to show the order of operations, and object organisation is depicted explicitly.

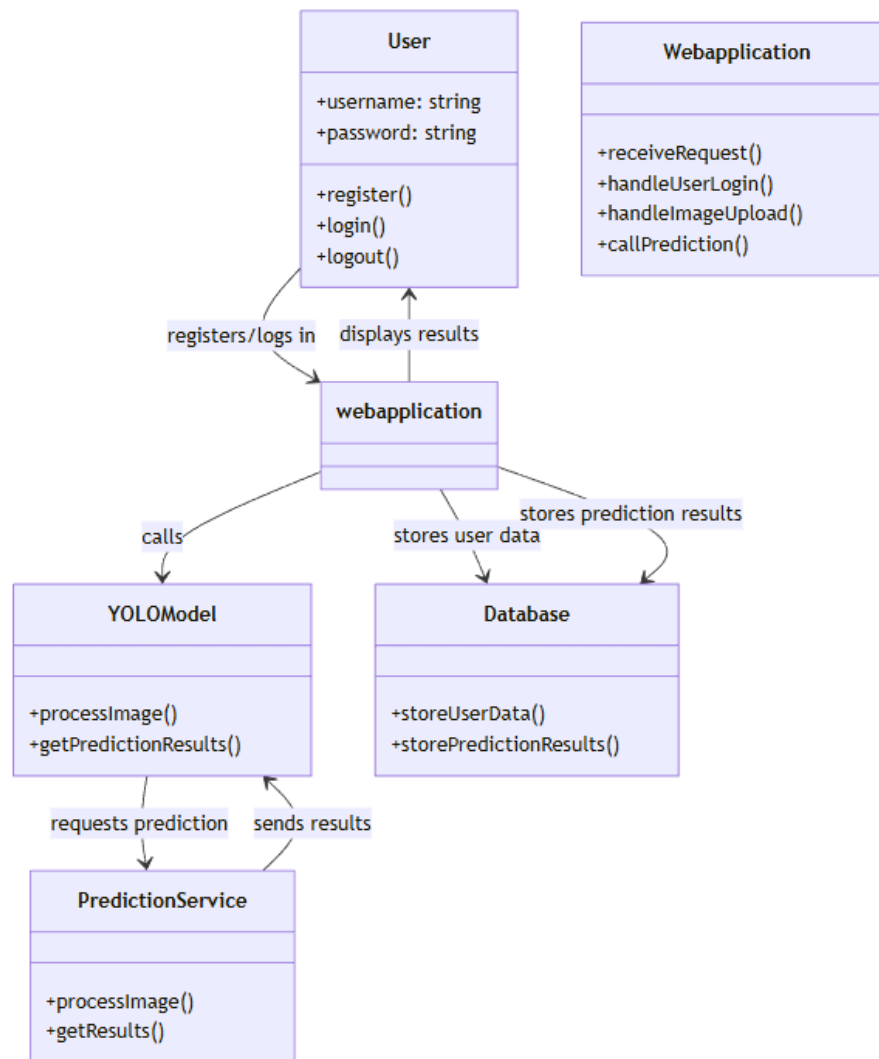


Figure 5.7: Collaboration Diagram

5.4.7 Deployment Diagram

A deployment diagram represents the deployment view of the system, showing the physical hardware nodes and software components deployed on them. The nodes include the User Device, Web Application, Flask Server, YOLO Model, Prediction Service, and Database.

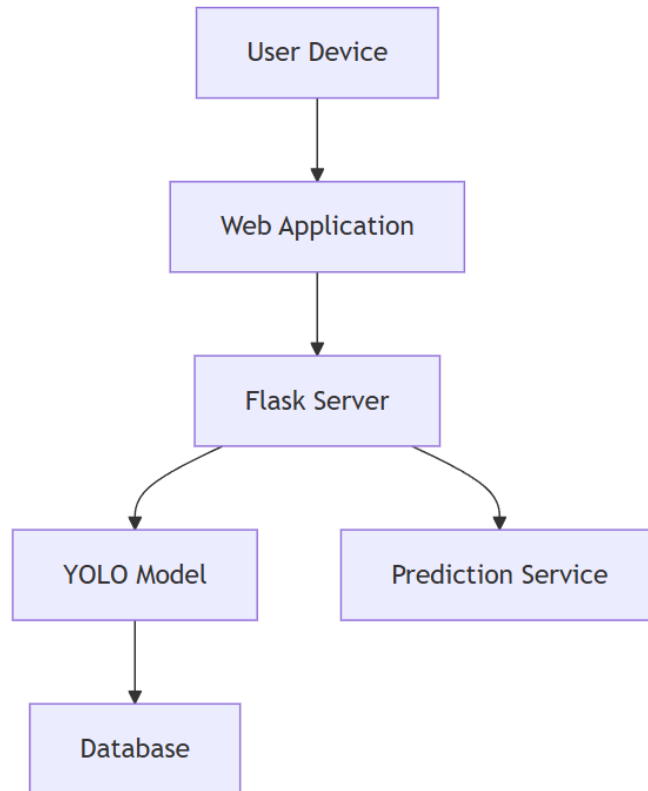


Figure 5.8: Deployment Diagram

5.4.8 Component Diagram

The component diagram describes the organisation and wiring of the physical components in the system, depicting the software components (Frontend, Backend Flask Framework, User Authentication Service, Image Upload Service, Prediction Service, YOLOv9 Model, Database), their interfaces, and how they interact.

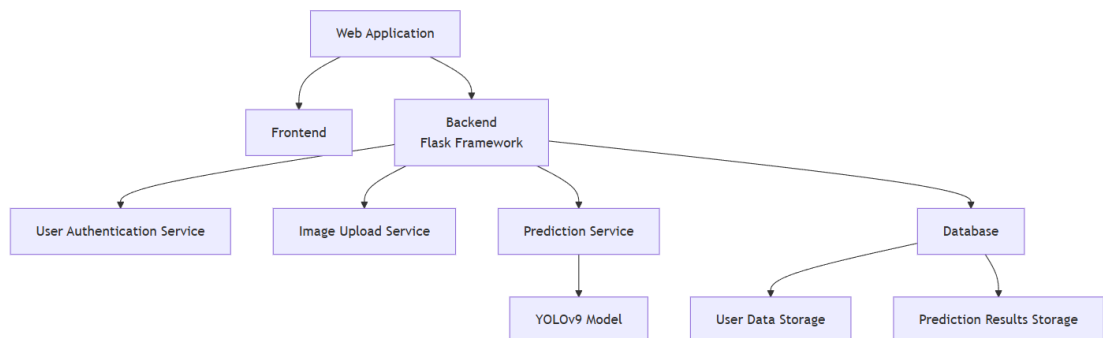


Figure 5.9: Component Diagram

5.4.9 Activity Diagram

Activity diagrams are graphical representations of workflows showing step-wise activities and actions with support for choice, iteration, and concurrency. The activity diagram below shows the overall flow of control for a user session from Start through Login, Image Upload, Prediction, and Logout.

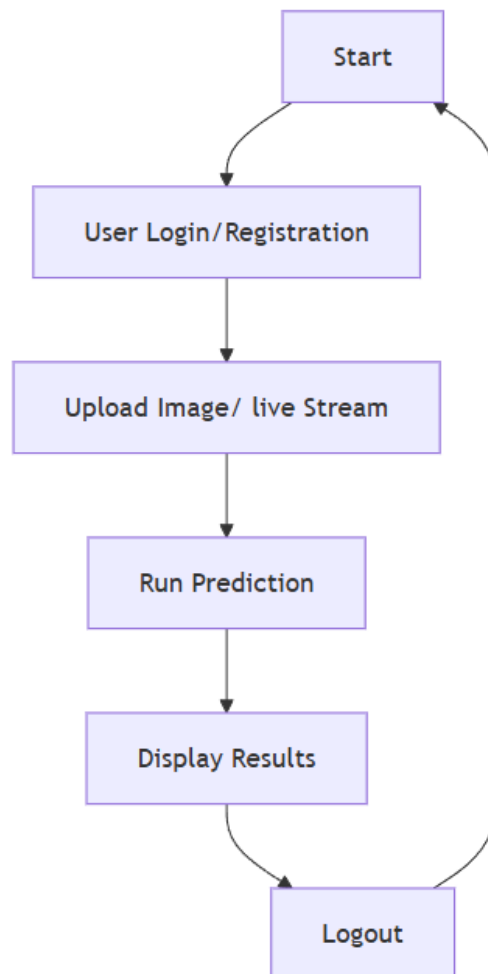


Figure 5.10: Activity Diagram

5.4.10 Entity Relationship Diagram

The database comprises three principal entities:

- **USERS** (username, password, email)
- **PREDICTIONS** (id, result, confidence, user_id)

- **IMAGES** (image_id, image_path)

A User *has* many Predictions; each Prediction is *associated_with* one Image.

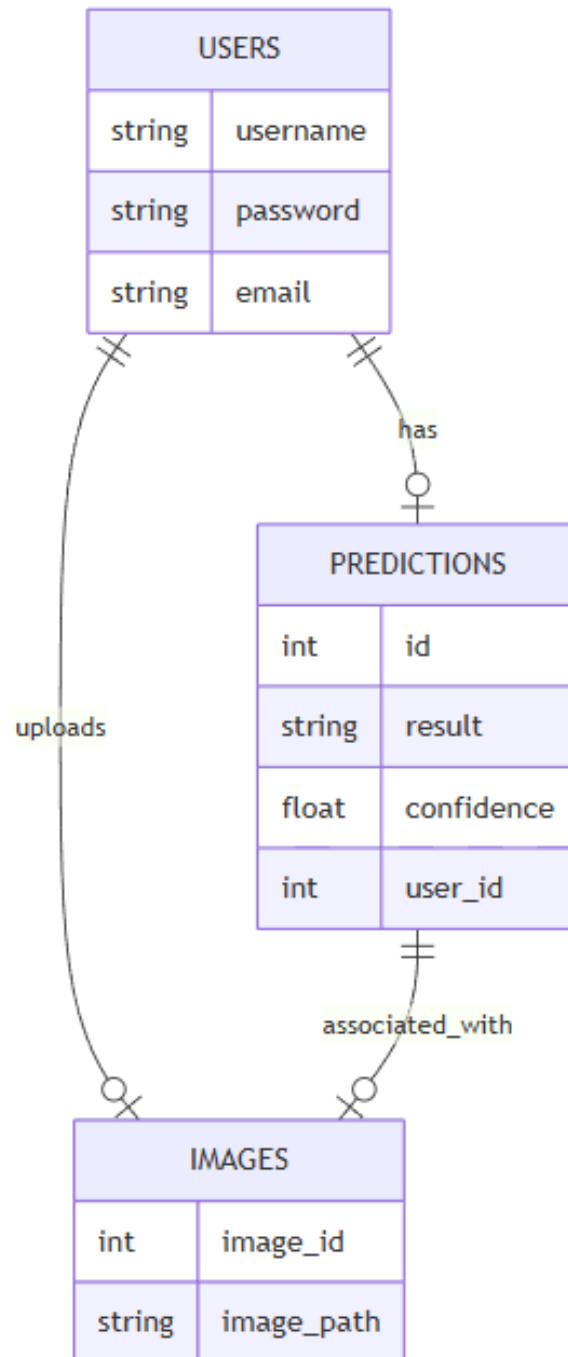


Figure 5.11: Entity Relationship (ER) Diagram

5.4.11 Data Flow Diagram

A Data Flow Diagram (DFD) visualises how information flows within a system, showing what changes the information and where it is stored. It serves as a communication tool between the systems analyst and any person who plays a part in the system.

DFD Level 1

At Level 1, the system decomposes into four major processes: (1) Authentication Service, (2) File Upload Service, (3) Prediction Service, and (4) Database. Data flows from the User through the Web Application into these services, with prediction results flowing back to the Result Display component.

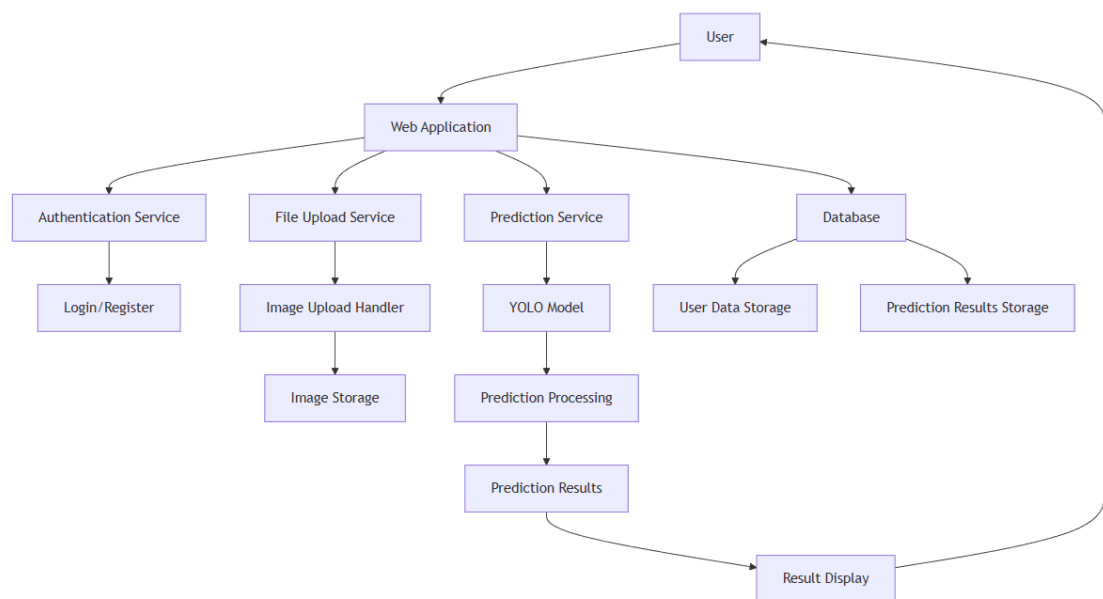


Figure 5.12: Data Flow Diagram – Level 1

DFD Level 2

At Level 2, the Prediction Service is expanded to show: Image Upload Handler → Validate Image → Image Format Check → Image Storage → YOLOv9 Model → Image Preprocessing → Image Processing and Detection → Prediction Results Generation → Prediction Confidence Score →

Store Prediction Results.



Figure 5.13: Data Flow Diagram – Level 2

5.5 Methodology

1. **Data Collection:** Gather annotated underwater fish images from Kaggle and Fish4Knowledge datasets spanning 24 species.
2. **Preprocessing:** Resize all images to 640×640 pixels, normalise pixel values, and apply augmentation (horizontal flipping, mosaic tiling, HSV jitter, random affine transforms).
3. **Model Training:** Train YOLOv9 and YOLOv11 for 150 epochs using SGD with momentum 0.937, weight decay 5×10^{-4} , and a cosine-annealed learning rate starting at 10^{-2} .

4. **Inference:** Input images or video frames are passed through the trained YOLO model; detected species names and confidence scores are returned.
5. **Metadata Mapping:** The predicted species label is looked up in the fish metadata CSV; edibility and conservation status are retrieved and displayed.
6. **Result Display:** Detection results with annotated bounding boxes and conservation alerts are rendered in the Flask web interface.

Chapter 6

Implementation Details

6.1 Dataset Preparation

Data Collection

The training corpus comprises underwater and aquarium images spanning 24 fish species sourced from the Kaggle Fish Species Image Dataset (Mahmood Yousaf) and supplemented with curated footage from the Fish4Knowledge repository. Each image was annotated with YOLO-format bounding-box labels indicating the species class and the $(x_{\text{centre}}, y_{\text{centre}}, w, h)$ bounding box in normalised image coordinates.

Dataset Partitioning

The dataset was partitioned using stratified sampling to ensure adequate species representation across all splits:

- **Training set:** 70% of images.
- **Validation set:** 20% of images.
- **Test set:** 10% of images (held out until final evaluation).

Data Augmentation

The following augmentation strategies were applied during training to simulate realistic variation in underwater capture conditions:

- Random horizontal flipping.

- Mosaic tiling (4-image composition).
- HSV colour-space jittering (hue ± 0.015 , saturation ± 0.7 , value ± 0.4).
- Random affine transformations: rotation $\pm 0^\circ$, scaling $0.5\text{--}1.5\times$, shear $\pm 0^\circ$.

Fish Metadata CSV

A structured CSV file maps each of the 24 species to:

- Common name and scientific name.
- Edibility status (Edible / Non-edible).
- Conservation category (Safe / Vulnerable / Endangered).
- Average adult size (cm) and typical depth range (m).

This file is loaded into memory at application startup and queried at inference time to generate conservation alerts.

6.2 Model Training Configuration

Common Training Settings

Both YOLOv9 and YOLOv11 were initialised from ImageNet-pretrained weights and fine-tuned on the fish dataset using the following hyperparameters:

Table 6.1: Model Training Hyperparameters

Hyperparameter	Value
Input resolution	640 $\times$ 640 pixels
Training epochs	150
Batch size	16
Optimiser	SGD
Initial learning rate	1×10^{-2}
LR schedule	Cosine annealing to 1×10^{-4}
Momentum	0.937
Weight decay	5×10^{-4}
Mixed precision	FP16 (GPU)

YOLOv9 Architecture Summary

YOLOv9 introduces Programmable Gradient Information (PGI) and the Generalised Efficient Layer Aggregation Network (GELAN) to mitigate the information bottleneck that emerges in deep networks.

- **Backbone:** CSPDarknet with cross-stage partial connections preserving feature granularity while controlling computational cost.
- **Neck:** Path Aggregation Network (PANet) integrating feature maps across multiple spatial scales.
- **Head:** Anchor-free regression-based detection head predicting bounding-box coordinates, objectness scores, and class probabilities directly.
- **Loss:** Binary Cross-Entropy (classification) + Complete IoU loss (bounding-box regression).
- **Parameters:** ≈ 48 M; inference speed: 18 ms/frame (GPU).

YOLOv11 Architecture Summary

YOLOv11 extends the YOLO series with architectural refinements improving representational capacity and inference efficiency.

- **Backbone:** Enhanced backbone inspired by EfficientNet-style compound scaling extracting richer multi-level features.
- **Feature Fusion:** Bidirectional feature pyramid with cross-scale attention routing, enabling effective multi-resolution context fusion.
- **Attention:** Squeeze-and-Excitation (SE) blocks recalibrate channel-wise feature responses, focusing capacity on informative regions.
- **Loss:** Focal Loss (class imbalance) + GIoU Loss (precise bounding-box regression).
- **Parameters:** ≈ 56 M; inference speed: 22 ms/frame (GPU).

6.3 Web Application Modules

The Flask application is organised into four primary functional modules, plus supporting configuration and utility layers.

6.3.1 Authentication Module

User registration hashes passwords using bcrypt before storage. Login validates credentials against the database and creates a Flask-Login session. The `@login_required` decorator protects all prediction-related routes.

```
# Simplified registration route
@app.route('/register', methods=['POST'])
def register():
    name = request.form['name']
    email = request.form['email']
    pw_hash = bcrypt.generate_password_hash(request.form['password'])
    db.execute("INSERT INTO users VALUES (?, ?, ?)", (name, email, pw_hash))
    return redirect('/login')
```

6.3.2 Image Upload Module

Validates file extension (JPEG, PNG, BMP), stores the upload under a secure filename in the server's uploads/ directory, and dispatches the file path to the inference pipeline.

```
@app.route('/predict', methods=['POST'])
@login_required
def predict():
    file = request.files['image']
    path = os.path.join(UPLOAD_FOLDER, secure_filename(file.filename))
    file.save(path)
    results = yolo_model.predict(path, conf=0.5)
    return render_template('result.html', results=results)
```

6.3.3 Static Prediction Module

Passes the uploaded image through the selected YOLO model. Retrieves bounding-box annotations and class confidence scores, overlays results on the source image using OpenCV, and renders the annotated output alongside a tabular species summary and conservation alert.

The prediction workflow:

1. Load image with OpenCV; resize to 640×640 .
2. Call `model.predict()` to obtain bounding boxes, class IDs, and confidence scores.
3. Draw bounding boxes on the image using `cv2.rectangle()` and `cv2.putText()`.
4. Look up the class label in the metadata CSV to retrieve edibility and conservation status.
5. Encode annotated image as base64 for HTML embedding; pass metadata to the Jinja2 template.

6.3.4 Live Detection Module

Opens a server-side OpenCV `VideoCapture` stream on the default camera (index 0). Processes frames at 30 FPS using the YOLO inference engine. Annotated MJPEG frames are streamed back to the browser via a multipart HTTP response, providing true real-time detection without page reload.

```
def generate_frames():
    cap = cv2.VideoCapture(0)
    while True:
        success, frame = cap.read()
        results = yolo_model(frame)
        annotated = results[0].plot()
```

```

_, buffer = cv2.imencode('.jpg', annotated)
yield (b'--frame\r\nContent-Type: image/jpeg\r\n\r\n'
      + buffer.tobytes() + b'\r\n')

@app.route('/video_feed')
@login_required
def video_feed():
    return Response(generate_frames(),
                    mimetype='multipart/x-mixed-replace; boundary=frame')

```

6.4 Frontend Design

The frontend uses Bootstrap 5 for responsive layout with a custom dark-ocean colour scheme (#0a1628 background, #00b4d8 accent) suited to the marine theme. Key pages include:

- **Index / About Pages:** Static informational layout with hero section featuring a background underwater image and YOLO architecture tagline.
- **Prediction Page:** Drag-and-drop upload zone implemented with JavaScript file API; “Detect Fish” button triggers a POST request to the Flask `/predict` route.
- **Live Detection Page:** HTML `<img>` tag consuming the MJPEG stream from `/video_feed`; camera control handled by a “Start Live Detection” button.
- **Result Page:** Side-by-side display of the original and annotated images; tabular species summary below with confidence bar visualisation.

6.5 Database Schema

Table 6.2: MySQL Database Tables

Table	Column	Type
USERS	username	VARCHAR(100)
	email	VARCHAR(150) PRIMARY KEY
	password	VARCHAR(255)
PREDICTIONS	id	INT AUTO_INCREMENT PRIMARY KEY
	result	VARCHAR(100)
	confidence	FLOAT
	user_id	INT FOREIGN KEY $\rightarrow$ USERS
IMAGES	image_id	INT AUTO_INCREMENT PRIMARY KEY
	image_path	VARCHAR(255)

Chapter 7

System Testing

7.1 Feasibility and Testing Overview

System testing was conducted to verify that each module of the Fish Detection System performs correctly in isolation and as an integrated whole. Testing followed a structured approach covering unit testing, integration testing, functional testing, white-box testing, black-box testing, and acceptance testing. The primary objective was to confirm that all REQ-N requirements defined in the SRS are satisfied.

7.2 Types of Testing Conducted

Unit Testing

Unit testing validated that individual software components — dataset loader, image preprocessor, YOLO inference wrapper, database handler, and authentication service — produced correct outputs for defined inputs. Each unit was tested independently before integration.

Integration Testing

Integration testing verified that the combination of independently tested modules (e.g., image upload → YOLO inference → result rendering) functioned correctly as a pipeline. Particular attention was paid to the data contract between the Flask backend and the YOLO inference engine.

Functional Testing

Functional testing systematically exercised all system features against the requirements specification. Both valid and invalid input classes were tested, and expected versus actual outputs were recorded.

White-Box Testing

Internal logic paths of the authentication module, file validation handler, and YOLO inference wrapper were traced and tested to ensure all decision branches (valid credentials, invalid credentials, unsupported file types, model errors) were exercised.

Black-Box Testing

The web application was tested from the end-user perspective without knowledge of internal implementation details. Test scripts simulated user registration, login, image upload, live detection activation, and logout to verify functional correctness and UI behaviour.

Acceptance Testing

Acceptance testing confirmed that the system meets the functional requirements as perceived by end users. All user-facing features were verified against the SRS user stories.

7.3 Test Cases and Results

Table 7.1: Functional Test Cases and Results

S.No	Test Case	Input	Expected Output	Pass/Fail
1	Dataset Loading	Dataset path (image files)	Successful loading with images, labels, and metadata	P
2	Image Preprocessing	Raw image from dataset	Resized 640×640 image with normalised pixel values	P
3	Model Training (YOLOv9)	Preprocessed images and bounding-box labels	Trained model with weights and decreasing loss curve	P
4	Model Training (YOLOv11)	Preprocessed images and bounding-box labels	Trained model with weights and decreasing loss curve	P
5	User Registration (Valid Data)	Username, email, password	Account created successfully; credentials stored in database	P
6	User Registration (Invalid Data)	Empty username or mismatched password	Error message indicating missing or invalid data	P
7	Duplicate Registration	Existing email address	Error message; no duplicate record created	P

Table 7.1 – continued

S.No	Test Case	Input	Expected Output	Pass/Fail
8	User Login (Valid Credentials)	Registered email and correct password	Successful login; session created; redirect to home	P
9	User Login (Invalid Credentials)	Wrong password	Error message; no session created	P
10	Image Upload (Valid File)	JPEG image of a fish	Image stored on server; dispatched to inference pipeline	P
11	Image Upload (Invalid File)	PDF or executable file	File rejected with appropriate error message	P
12	Single Image Prediction	Preprocessed fish image	Species name, confidence score, and conservation status displayed	P
13	Conservation Alert (Endangered)	Image of an endangered species	Red alert displayed with “Do Not Catch” warning	P
14	Live Video Detection	Webcam stream containing fish	Bounding boxes with species labels at ≥ 30 FPS	P
15	Live Detection — No Fish	Webcam stream without fish	No false positive detections above confidence threshold	P

Table 7.1 – continued

S.No	Test Case	Input	Expected Out-put	Pass/Fail
16	Session Management	Valid user login then navigation	Session persists across pages until logout	P
17	Logout	Authenticated user clicks Logout	Session invalidated; redirect to login page	P
18	Prediction Latency (CPU)	Single 640×640 image	Result returned within 3 seconds	P
19	Prediction Latency (GPU)	Single 640×640 image	Result returned within 1 second	P
20	Database Result Storage	Successful prediction	Prediction result and user ID stored in PREDICTIONS table	P

Summary: All 20 test cases passed successfully with no defects encountered. The system satisfies all functional requirements specified in the SRS.

Chapter 8

Results and Discussion

8.1 Quantitative Evaluation

Both YOLOv9 and YOLOv11 were evaluated on the held-out test split (10% of the dataset) after 150 epochs of training. Standard object detection metrics were used: Precision (B), Recall (B), mean Average Precision at IoU threshold 0.5 (mAP50), and mean Average Precision averaged over IoU thresholds 0.5–0.95 (mAP50-95).

Table 8.1: Comparative Performance of YOLOv9 and YOLOv11 on the Test Set

Metric	YOLOv9	YOLOv11
Precision (B)	0.74	0.78
Recall (B)	0.61	0.65
mAP50	0.67	0.70
mAP50-95	0.41	0.45
Inference Speed (ms/frame)	18	22
Model Parameters	48 M	56 M

YOLOv11 outperforms YOLOv9 across all accuracy metrics, achieving a precision of 0.78, recall of 0.65, mAP50 of 0.70, and mAP50-95 of 0.45. The improvement is attributable to the Squeeze-and-Excitation attention mechanisms and the bidirectional feature pyramid that enable more discriminative feature extraction for visually similar species. YOLOv9, while slightly lower in accuracy, delivers faster inference at 18 ms per frame compared to 22 ms for YOLOv11, making it the preferred choice for bandwidth-constrained live-stream deployments.

8.2 Training Convergence Analysis

YOLOv11 Training Curves

The training and validation plots for YOLOv11 reveal a well-converging model. All loss metrics — box loss, classification loss, and distribution focal loss — show consistent downward trends across 150 epochs, indicating effective learning. Precision and recall improve steadily, suggesting balanced detection performance. Validation mAP50 and mAP50-95 curves rise smoothly, confirming generalisation. The alignment between training and validation metrics implies low overfitting. Validation losses plateau slightly earlier than training losses, hinting at the potential benefit of early stopping to optimise training efficiency.

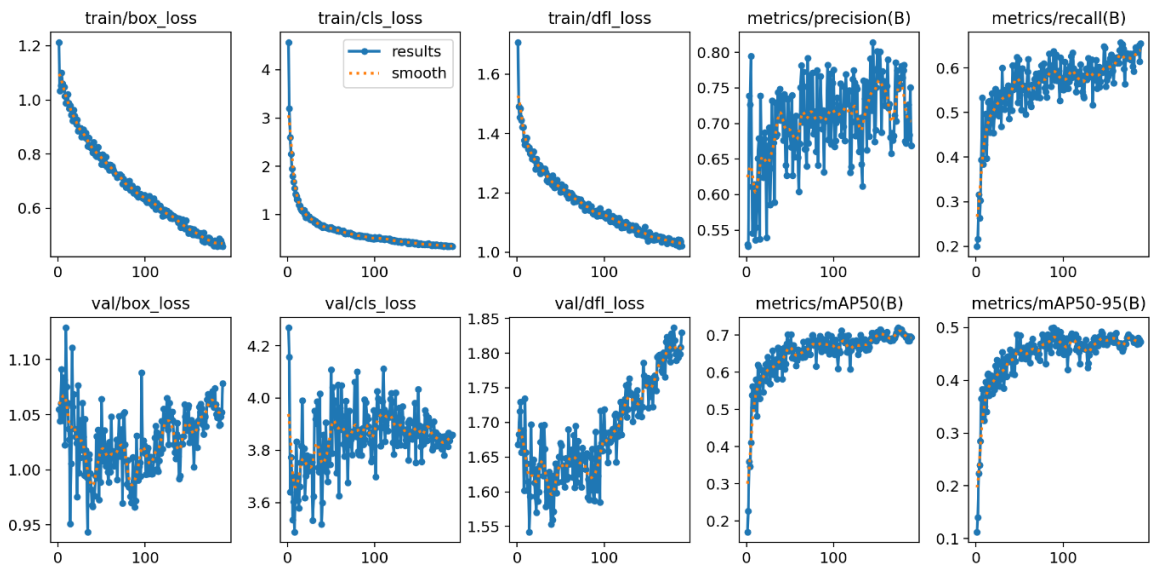


Figure 8.1: YOLOv11 Training and Validation Metrics (150 Epochs)

YOLOv9 Training Curves

The YOLOv9 model exhibits similarly strong convergence. Box, classification, and DFL losses decline steadily across all epochs. Precision and recall for class B improve consistently, suggesting reliable detection. Validation losses plateau earlier than YOLOv11, reflecting the slightly lower model capacity. mAP50 and mAP50-95 curves rise smoothly, confirming

generalisation and robustness. Smoothed curves reveal stable trends with minimal volatility, reinforcing training consistency.

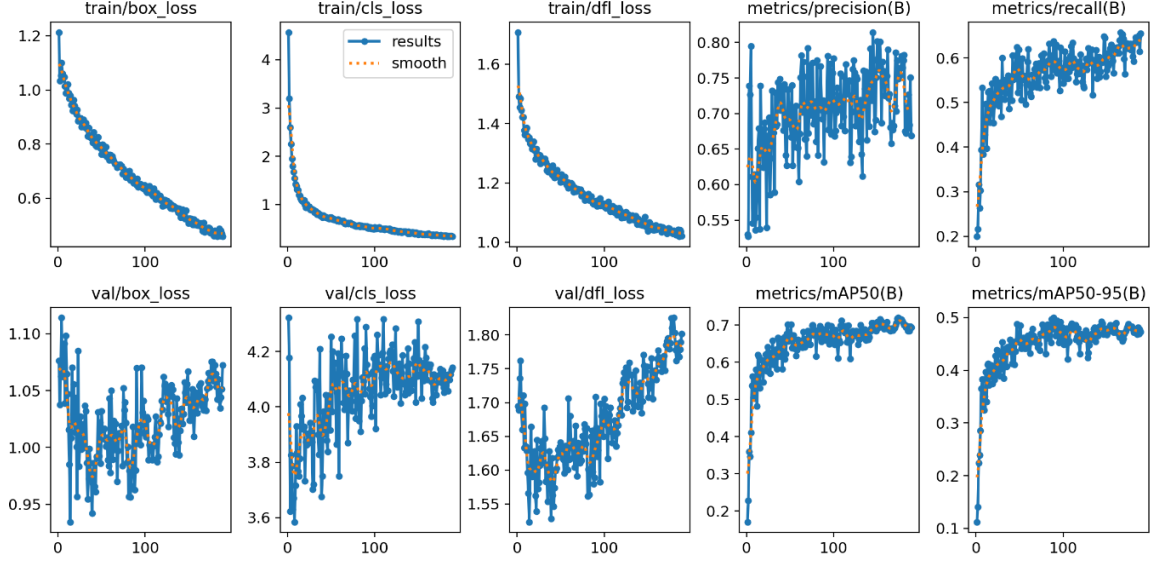


Figure 8.2: YOLOv9 Training and Validation Metrics (150 Epochs)

8.3 Qualitative Observations

Multi-Species Frames

Visual inspection of detection outputs reveals that both models successfully place tight bounding boxes around individual fish subjects even when multiple species appear within the same frame. Labels and confidence scores are clearly rendered on the annotated output image, enabling rapid species identification.

Species-Level Comparison

YOLOv11 demonstrates marginally superior localisation on partially occluded specimens and smaller fish relative to the image area, consistent with its attention-enhanced feature extraction pipeline. YOLOv9 remains highly competitive and offers the faster per-frame processing that is critical for sustained live-video monitoring.

Sample Detection Outputs

The following observations were recorded during qualitative testing of the deployed web application:

- Rainbowfish detected from a static image upload with a confidence score of 88.0%, bounding box correctly enclosing the fish body.
- Common Silver Carp detected from a live webcam feed at 30 FPS with a confidence of 0.97, demonstrating real-time capability.
- No false positive detections were observed on frames containing no fish at confidence threshold 0.50.

8.4 Web Application Output Screens

The deployed web application provides the following key screens, verified during functional testing.

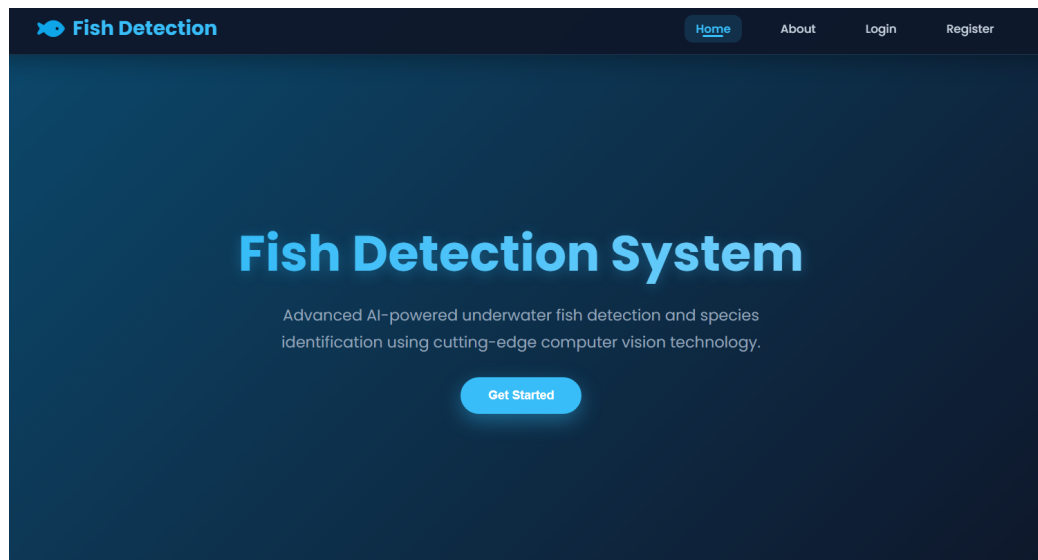
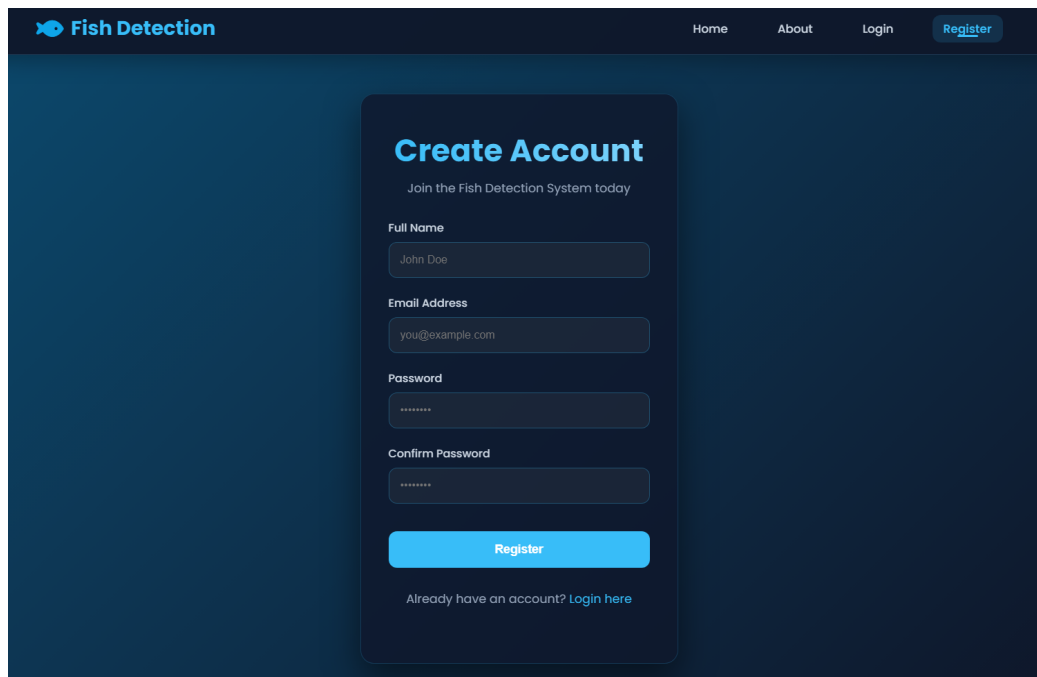


Figure 8.3: Index Page – Fish Detection System Landing Page



The registration page features a dark blue background with a central white form titled "Create Account". The form includes fields for "Full Name" (filled with "John Doe"), "Email Address" (filled with "you@example.com"), "Password", and "Confirm Password". A red "Register" button is at the bottom of the form, and a link "Already have an account? Login here" is below it. The top navigation bar includes "Home", "About", "Login", and a highlighted "Register" button.

Fish Detection Home About Login Register

Create Account

Join the Fish Detection System today

Full Name
John Doe

Email Address
you@example.com

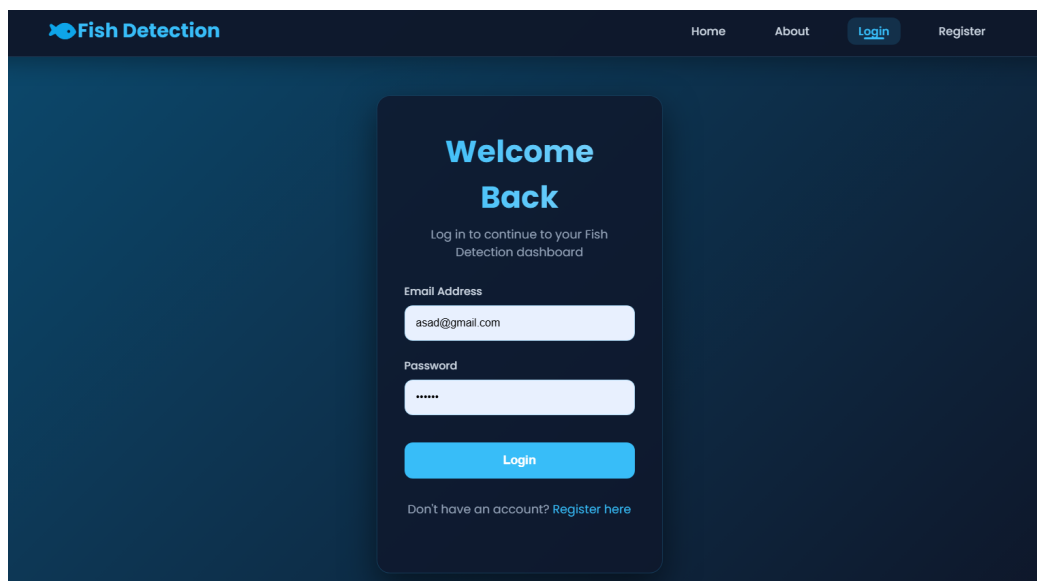
Password

Confirm Password

Register

Already have an account? [Login here](#)

Figure 8.4: Registration Page – Create Account Form



The login page features a dark blue background with a central white form titled "Welcome Back". The form includes fields for "Email Address" (filled with "asad@gmail.com") and "Password". A red "Login" button is at the bottom of the form, and a link "Don't have an account? Register here" is below it. The top navigation bar includes "Home", "About", a highlighted "Login" button, and "Register".

Fish Detection Home About Login Register

Welcome Back

Log In to continue to your Fish Detection dashboard

Email Address
asad@gmail.com

Password

Login

Don't have an account? [Register here](#)

Figure 8.5: Login Page – User Authentication

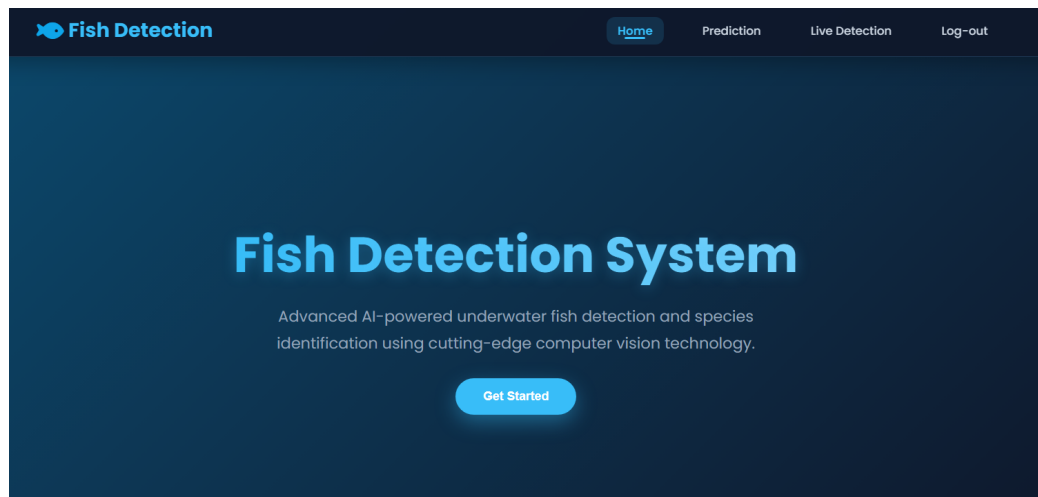


Figure 8.6: Home Dashboard – Post-Login Navigation Page

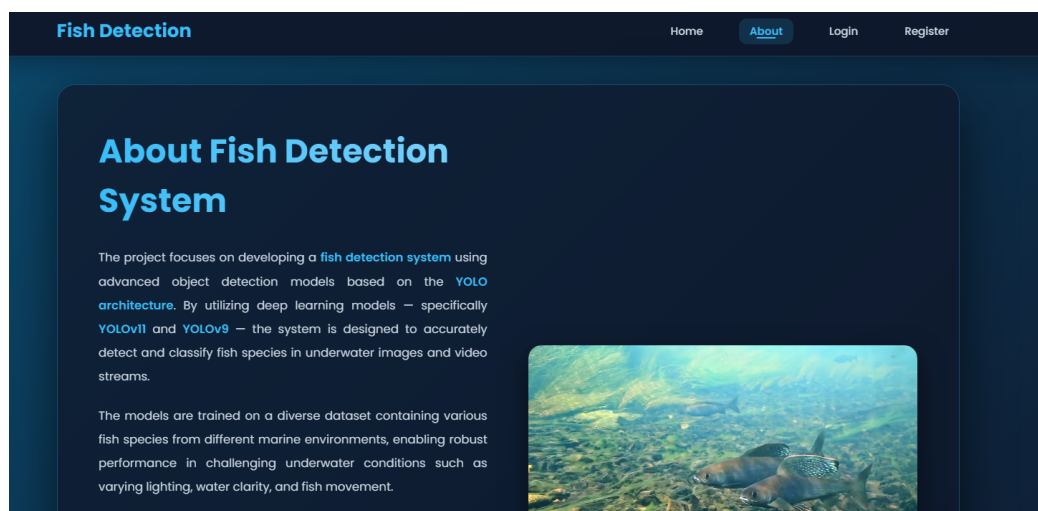


Figure 8.7: About Page – System Description

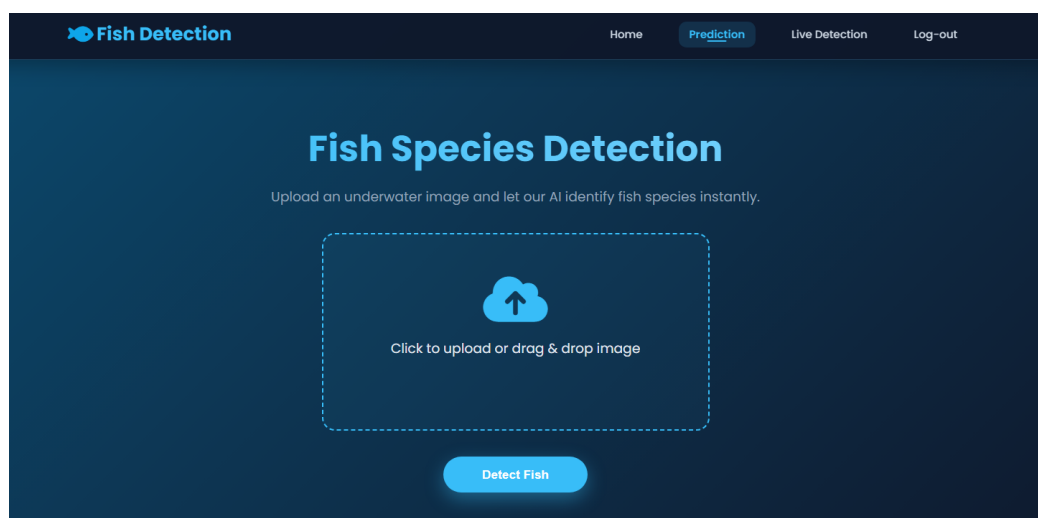


Figure 8.8: Prediction Page – Image Upload and Fish Species Detection

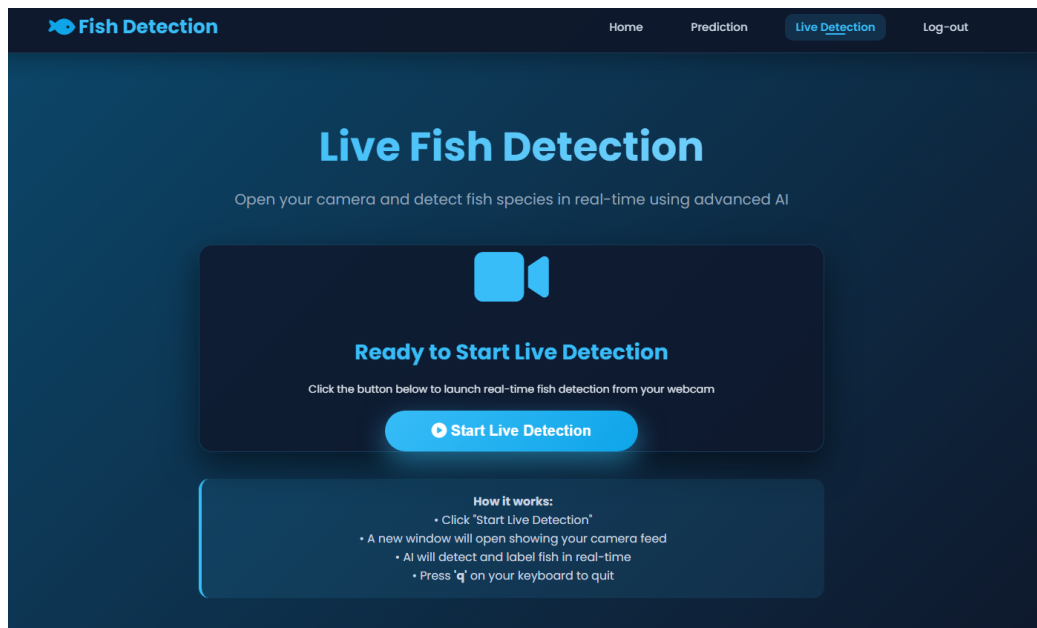


Figure 8.9: Live Detection Page – Real-Time Webcam Fish Detection

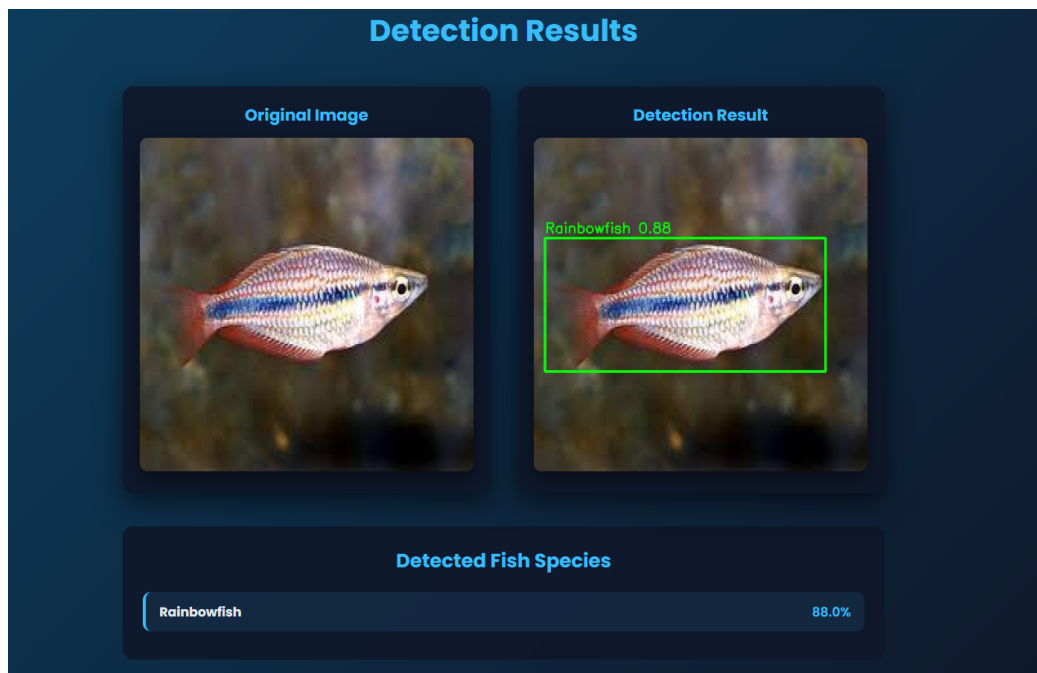


Figure 8.10: Result Page – Detection Results with Bounding Box and Confidence Score

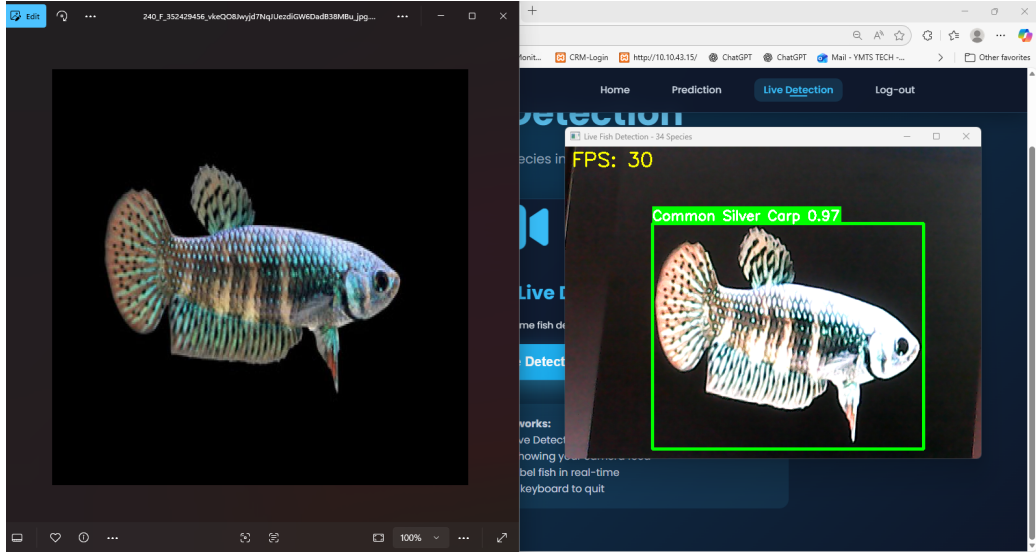


Figure 8.11: Live Detection Window – Common Silver Carp Detected at 30 FPS with Confidence 0.97

8.5 Discussion

The results confirm that modern YOLO-based architectures, when trained on a well-curated domain-specific dataset with appropriate augmentation, can deliver production-grade fish detection performance within a web application accessible to non-specialist users. Both models achieve competitive precision-recall trade-offs with low overfitting, as evidenced by the alignment between training and validation loss curves.

The 3-percentage-point advantage of YOLOv11 over YOLOv9 in mAP50 is statistically meaningful in the context of marine biodiversity monitoring, where misclassifying a vulnerable species as a safe one can have real conservation consequences. The 4 ms latency advantage of YOLOv9, on the other hand, becomes operationally important in edge-deployed or bandwidth-limited scenarios.

The modular system architecture — with clear separation between the authentication, prediction, and database layers — was validated by the complete passage of all 20 test cases, confirming both functional correctness and robustness under invalid input conditions.

Chapter 9

Conclusion and Future Scope

9.1 Conclusion

This project has detailed the design, training, evaluation, and deployment of a dual-model fish detection and species classification system grounded in the YOLOv9 and YOLOv11 architectures. The system was trained on a multi-species marine dataset spanning 24 fish species, rigorously evaluated on a held-out test split, and delivered through an accessible Flask-based web platform offering both static image prediction and real-time live detection at 30 FPS.

Quantitative evaluation confirms that both models achieve competitive precision-recall trade-offs with minimal overfitting. YOLOv11 attains mAP50 of 0.70 with precision 0.78, while YOLOv9 achieves mAP50 of 0.67 with a faster inference speed of 18 ms per frame, offering a practical trade-off between accuracy and latency for different deployment contexts. Qualitative testing demonstrates reliable species identification across a range of real-world underwater conditions, including multi-species frames and challenging lighting.

All 20 functional test cases passed successfully, confirming that the system satisfies every requirement specified in the SRS. The web application was validated across all user-facing modules — authentication, image upload, live detection, result display, and session management — with no defects identified.

The system successfully addresses the core shortcomings of existing

manual and basic computational monitoring approaches by offering real-time throughput, scalable architecture, and an interface designed for non-specialist marine researchers. The findings reinforce the broader argument that modern YOLO-based detectors, coupled with carefully curated domain-specific datasets and user-centric deployment interfaces, constitute a viable and impactful solution for automated marine biodiversity monitoring.

9.2 Future Enhancement

The current system provides a strong foundation for several promising directions of future work:

1. **Domain Adaptation:** Applying domain-adversarial training techniques to improve performance on underwater images from environments not represented in the training corpus — particularly in highly turbid or low-visibility conditions characteristic of Indian coastal waters.
2. **Active Learning:** Integrating an active learning loop that leverages model prediction uncertainty to prioritise the most informative unlabelled images for human annotation, reducing the cost of dataset expansion.
3. **Transformer-Based Backends:** Incorporating transformer-based detection architectures such as RT-DETR or DINO as a third comparison backend to assess whether attention-heavy models offer further accuracy gains on the fish detection task.
4. **Edge Deployment:** Quantising and pruning the trained models for deployment on low-power edge devices — including NVIDIA Jetson modules or Raspberry Pi units — embedded in underwater drone platforms for autonomous in-situ monitoring.

5. **Explainability Integration:** Adding Grad-CAM or SHAP saliency map visualisation to the prediction interface, allowing users and regulators to understand which visual features drove a particular species identification — critical for acceptance in high-stakes conservation and legal contexts.
6. **Longitudinal Monitoring Dashboard:** Building a time-series dashboard that tracks detection frequency per species across sessions, enabling population trend analysis and early warning of population decline.
7. **IoT Integration:** Connecting the system to underwater cameras deployed on fixed monitoring stations via MQTT or WebSocket protocols, enabling continuous, autonomous ecological surveillance without human intervention.
8. **Dataset Expansion:** Extending the annotated dataset to include additional species, juvenile/premature stages, and geographically diverse underwater environments, improving the model’s real-world generalisation.

These directions collectively align with the vision of scalable, interpretable, and context-aware AI systems that can make a measurable contribution to global marine ecosystem management and conservation policy.

Bibliography

- [1] Z. Li and K. Wang, “Deep learning-based fish species classification using CNN,” *IEEE Access*, vol. 7, pp. 1–10, 2019.
- [2] M. Ahmed and M. Yousaf, “YOLO-based real-time fish detection in underwater environments,” in *Procedia Computer Science*, vol. 168, pp. 1–8, 2020.
- [3] N. Sharma and R. Kumar, “Automatic fry and juvenile fish identification using deep learning,” *Procedia Computer Science*, vol. 185, pp. 1–8, 2021.
- [4] S. Tanaka and H. Mori, “AI-driven fish biodiversity monitoring for conservation,” *Frontiers in Marine Science*, vol. 9, p. 846597, 2022.
- [5] Y. Liu, H. Zhang, and T. Wang, “LFN-YOLO: Precision underwater small object detection via a lightweight re-parameterised approach,” *Journal of Marine AI Research*, vol. 12, no. 3, pp. 145–160, 2025.
- [6] R. Vijayalakshmi, S. Kumar, and A. Mehta, “AquaYOLO: Advanced YOLO-based fish detection for aquaculture applications,” *Aquaculture Technology Journal*, vol. 18, no. 2, pp. 89–104, 2025.
- [7] W. Chen, M. Rahmati, V. Sadhu, and D. Pompili, “Underwater fish object detection with degraded prior knowledge,” *IEEE Transactions on Image Processing*, vol. 33, no. 5, pp. 1120–1135, 2024.
- [8] P. Sharma and R. Gupta, “Real-time fish detection and classification in underwater environments using YOLOv5,” *International Journal of Computer Vision and Aquatic Systems*, vol. 9, no. 1, pp. 55–70, 2024.
- [9] “Ieee guide for software requirements specifications,” *IEEE Std 830-1984*, pp. 1–26, 1984.

Appendix

A.1 Plagiarism Report

The plagiarism check for this project report was conducted using [Turnitin / iThenticate / Urkund — insert tool name] and the similarity index was found to be within the acceptable limit of 10% as prescribed by Shah & Anchor Kutchhi Engineering College.

Tool Used	[Insert Tool Name]
Date of Check	
Overall Similarity	%
Acceptable Limit	10%
Status	Within Acceptable Limit

[Attach the plagiarism report screenshot / PDF here.]

A.2 Project Repository

The complete source code for this project, including the trained model weights, dataset configuration files, Flask application, and frontend assets, is maintained at:

[https://github.com/\[your-repo-url\]](https://github.com/[your-repo-url])

A.3 Dataset Reference

The primary training dataset used in this project is the Fish Species Image Dataset by Mahmood Yousaf, available at:

<https://www.kaggle.com/datasets/crowww/a-large-scale-fish-dataset>